



Empowering Learners to Build Production - Ready
AI Agents

Applied Agentic AI for SWEs

Week 0 - Mini Project

Problem Statement

Hello Agent - CSV FAQ Agent

Week 0 Mini Project

Hello Agent - CSV FAQ Agent

Estimated time - About 60 to 90 minutes

Project status - This is a warm-up exercise. It is not graded, and it will not affect your progress or evaluation in this program. The goal is to help you become comfortable with the basic tools and ideas before we move to deeper projects. You are encouraged to try things, break things, and fix them again. You can change the prompt, upload different files, or ask unusual questions just to see what happens. The purpose is to understand the flow end-to-end rather than to produce a perfect product.

You can relax, experiment, and focus on understanding the basic idea of how an agent can sit on top of simple business data and answer questions from it.

1. Background and Story

Imagine you work as a junior AI engineer in a small online business.

The company has

- An e-commerce store that sells electronics and clothing
- A small credit card product with its own terms and conditions
- A hospital client that wants to show clear public policies on visits and payments
- A software as a service product with many features and limits

All the important answers for customers and internal staff already exist in simple CSV files

- Ecommerce frequently asked questions
- Credit card terms
- Hospital policies
- Software as a service product documentation

Support agents spend a lot of time reading these files and searching for the right row. They want a simple tool where they can ask normal questions and get clear answers from the CSV data.

Your task is to build the very first version of that tool.

2. Project Goal

Build a small web app that works as a **CSV FAQ Agent**

- The user can upload one or more CSV files
- The user can ask a question in normal language
- The AI reads the data in those CSV files and replies using real values from the tables

Important points

- The answer must come from the CSV content
- The model should not answer from general world knowledge
- If the answer is not in the data, it should say that clearly

This mini project will help you understand how agents can work with tabular data using a very simple design.

3. What You Will Learn

By the end of this mini project you will be able to

- Explain the idea of a FAQ agent that reads from CSV files
- Build a small Streamlit app that lets users upload CSV files
- Use a language model plus a data frame agent to answer questions from the data
- Understand how a system prompt can control model behavior
- Think in terms of a business requirement and then map it to technical steps

4. Business Requirement

4.1 Short description

We want a simple internal tool called **Hello Agent**.

Support agents should be able to

- Upload the latest CSV files for policies and FAQs

- Type a natural language question
- Get a clear answer that they can copy paste into an email or chat

This tool will reduce time spent searching in documents and will improve answer consistency.

4.2 Target users

- Customer support agents
- Product managers who want to test how clear their policy text is
- Operations staff who need quick answers while talking to customers on phone or chat

4.3 Data sources

For this mini project you will use sample CSV files - **Click here to Download**  Data sources

- [ecommerce_faqs.csv](#) - frequently asked questions for an online store
- [credit_card_terms.csv](#) - terms and conditions for credit cards
- [hospital_policy.csv](#) - hospital rules for visits, records, and other items
- [saas_docs.csv](#) - features, limits, and support details for a software as a service product

In a real company these CSV files may be exported from a knowledge base or a help desk system.

4.4 Functional requirements

Your app should cover the following points.

1. File upload
 - The user can upload one or many CSV files from their local machine
 - The app shows a small preview of each file, for example the first few rows
2. Question input
 - The user can type a question in a text box in normal language
 - Example questions
 - What is the return policy for electronics
 - What does the extended warranty cover
 - What are the visiting hours in the hospital
 - What is the API rate limit for the free plan

3. Answer generation
 - The AI must read the CSV data and find the most relevant row or rows
 - For text questions it should return the key piece of text, for example from the Answer or Policy column
 - For numeric questions it should compute totals or averages if needed
 - The answer should be in clear English, ready to share with a customer
4. Data only rule
 - The AI must not guess from general knowledge
 - If it cannot find a matching answer in the CSV data, it should say something like
 - I could not find this information in the uploaded files
5. Simple user interface
 - The interface should be simple and clean
 - A first time user should understand what to do without reading long instructions

4.5 Non functional expectations

This is a small warm up project, so expectations are simple

- The app should feel responsive on small test CSV files
- The code should be easy to read and change
- The behavior should be predictable and safe
 - No random creative answers
 - The model should mostly keep temperature low and follow rules

5. Technical Requirements

5.1 Tools and libraries

You will use

- Python
- Streamlit for the web app
- Pandas for reading and working with CSV files
- LangChain for building a data frame agent
- OpenAI chat model, for example `gpt 4o mini`
- A valid OpenAI API key

You do not need

- Any vector database

- Any complex retrieval system
- Any external database such as MySQL or MongoDB

5.2 Minimum expected behavior

Your app should

- Accept multiple CSV uploads
- Convert each CSV into a Pandas data frame
- Create a LangChain agent that can work with these data frames
- Build a system prompt that forces the model to use the data only
- Pass the user question plus the system prompt into the agent
- Display the answer clearly in the Streamlit app